

Chapitre 2 : Organisation Mémoire

Mots Machine, Adresses, Pointeurs, Tableaux et Chaînes

Dr. El Hadji Bassirou TOURÉ
Département de Mathématiques et Informatique
Faculté des Sciences et Techniques
Université Cheikh Anta Diop de Dakar

1 Le Mot Machine et l'Organisation de la Mémoire

1.1 Définition du Mot Machine

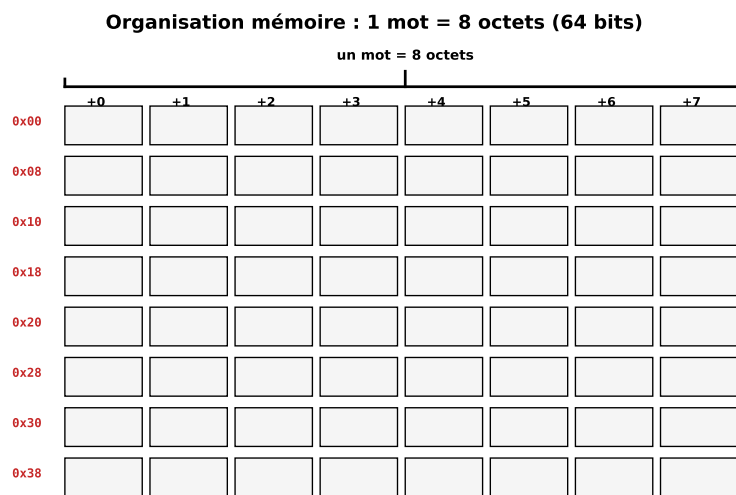
Mot machine (word)

Un **mot** (mot machine ou *word*) est l'unité de base manipulée par un microprocesseur. La taille d'un mot s'exprime en bits et sert à classer les architectures : 32 bits ou 64 bits. Cette taille détermine la largeur des registres du processeur, la taille des adresses mémoire, et la quantité de données traitées en un cycle.

La taille du mot machine a des implications directes sur les capacités du système. Sur une architecture 32 bits, les adresses sont codées sur 32 bits, ce qui limite l'espace d'adressage à $2^{32} = 4$ Go. Sur une architecture 64 bits, l'espace d'adressage théorique est de 2^{64} octets, bien que les systèmes actuels n'utilisent que 48 bits d'adresse (256 To).

1.2 Vue de la Mémoire par un Programme

Un programme machine perçoit la mémoire comme un très grand tableau linéaire d'octets appelé **mémoire virtuelle**. Chaque octet possède une adresse unique, et l'ensemble des adresses possibles constitue l'**espace d'adressage virtuel**.



Chaque case représente 1 octet (8 bits)
L'adresse de chaque ligne correspond au premier octet de cette ligne

Rappel Hexadécimal

Dec	Bin	Hex
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

2 Adressage et Représentation des Données

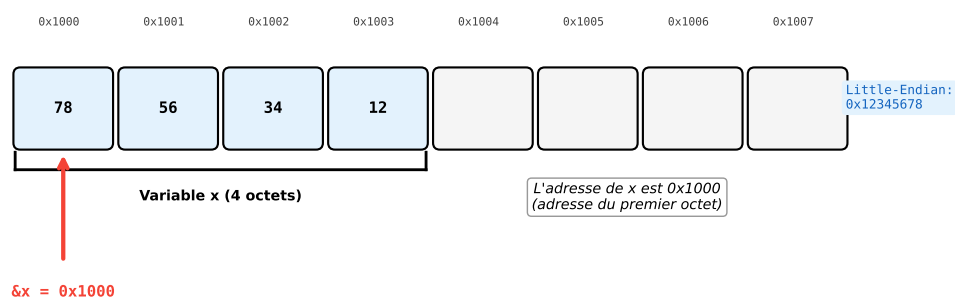
2.1 Concepts Fondamentaux de l'Adressage

Adresse et Zone Mémoire

- Une **adresse** est un nombre qui identifie de manière unique un octet en mémoire
- Lorsqu'un objet occupe plusieurs octets, son **adresse** est celle de son premier octet (l'octet à l'adresse la plus basse)
- Pour accéder à un objet en mémoire, on a besoin de deux informations : son **adresse** et sa **taille**

En C, l'opérateur `&` permet d'obtenir l'adresse d'une variable. Cette adresse est toujours celle du premier octet occupé par la variable. Par exemple, si une variable `int x` est stockée aux adresses `0x1000` à `0x1003`, alors `&x` vaut `0x1000`.

Adresse d'un objet multi-octets



2.2 Taille des Types selon l'Architecture

La taille des types primitifs en C dépend de l'architecture et du compilateur. Le standard C garantit seulement des tailles minimales et des relations d'ordre : `sizeof(char) ≤ sizeof(short) ≤ sizeof(int) ≤ sizeof(long)`.

Taille des types selon l'architecture

Type C	IA-32 (32 bits)	x86-64 (64 bits)	Plage de valeurs (64 bits)
char	1 octet	1 octet	-128 à 127
unsigned char	1 octet	1 octet	0 à 255
short	2 octets	2 octets	-32768 à 32767
int	4 octets	4 octets	$\approx \pm 2.1 \times 10^9$
long	4 octets	8 octets	$\approx \pm 9.2 \times 10^{18}$
long long	8 octets	8 octets	$\approx \pm 9.2 \times 10^{18}$
float	4 octets	4 octets	$\approx \pm 3.4 \times 10^{38}$
double	8 octets	8 octets	$\approx \pm 1.8 \times 10^{308}$
void *	4 octets	8 octets	adresse mémoire

Portabilité

La différence de taille de `long` entre 32 et 64 bits est source de bugs. Pour un code portable, utilisez les types de `<stdint.h>` : `int32_t`, `int64_t`, `uint8_t`, etc.

La taille d'un pointeur correspond **toujours** à la taille du mot machine : 4 octets sur 32 bits, 8 octets sur 64 bits. C'est logique : un pointeur stocke une adresse, et une adresse occupe exactement un mot machine.

3 Endianness : Ordre des Octets en Mémoire

3.1 Le Problème de l'Ordre des Octets

Lorsqu'une donnée occupe plusieurs octets (comme un `int` de 4 octets), une question se pose : dans quel ordre sont stockés ces octets en mémoire ? Deux conventions existent, appelées **endianness** (du roman *Les Voyages de Gulliver* de Jonathan Swift).

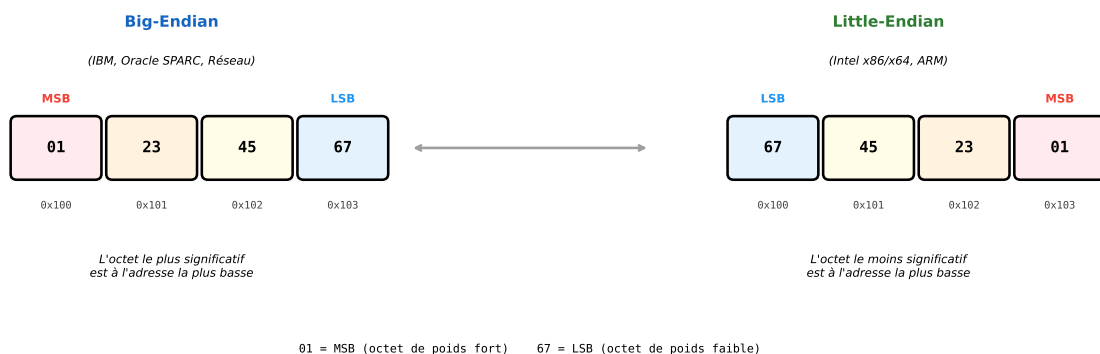
Little-Endian vs Big-Endian

- **Little-Endian** ("petit-boutiste") : l'octet de **poids faible** (LSB - Least Significant Byte) est stocké à l'adresse la plus basse
- **Big-Endian** ("gros-boutiste") : l'octet de **poids fort** (MSB - Most Significant Byte) est stocké à l'adresse la plus basse

3.2 Exemple Détaillé

Considérons la valeur hexadécimale `0x01234567` stockée à l'adresse `0x100`. Cette valeur se décompose en 4 octets : 01 (MSB), 23, 45, 67 (LSB).

Stockage de `0x01234567` à l'adresse `0x100`



3.3 Quand l'Endianness Importe

L'endianness devient critique dans trois situations :

1. **Communication réseau** : lorsqu'on envoie des données binaires entre machines de conventions différentes
2. **Lecture de fichiers binaires** : un fichier créé sur une machine big-endian sera mal lu sur une machine little-endian
3. **Réinterprétation mémoire** : lorsqu'on lit une zone mémoire avec un type différent de celui utilisé pour l'écrire

Listing 1 – Détection de l'endianness

```
1 #include <stdio.h>
2 #include <stdint.h>
3
4 int is_little_endian(void) {
5     uint32_t x = 1; /* En mémoire : 01 00 00 00 (LE) ou 00 00 00 01 (BE) */
6     uint8_t *p = (uint8_t *)&x;
7     return p[0] == 1; /* Si le premier octet vaut 1, c'est little-endian */
8 }
9
10 int main(void) {
11     if (is_little_endian())
12         printf("Cette machine est Little-Endian\n");
13     else
14         printf("Cette machine est Big-Endian\n");
15     return 0;
16 }
```

4 Les Pointeurs : Variables Contenant des Adresses

4.1 Définition et Déclaration

Pointeur

Un **pointeur** est une variable dont la valeur est une adresse mémoire. Le type du pointeur indique au compilateur comment interpréter les données à cette adresse et comment effectuer l'arithmétique des pointeurs.

La déclaration d'un pointeur utilise l'opérateur ***** :

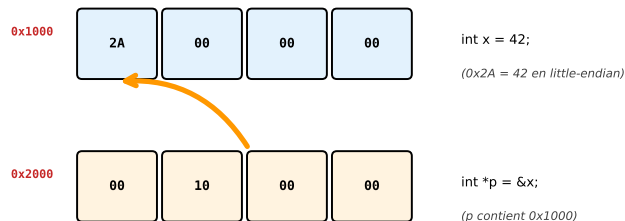
```
1 int x = 42;           /* Variable entière */
2 int *p;              /* Déclaration : p est un pointeur vers un int */
3 p = &x;              /* p contient maintenant l'adresse de x */
4
5 printf("x = %d\n", x); /* Affiche : x = 42 */
6 printf("&x = %p\n", &x); /* Affiche l'adresse de x */
7 printf("p = %p\n", p); /* Même adresse que &x */
8 printf("*p = %d\n", *p); /* Déréférencement : affiche 42 */
```

4.2 Les Deux Aspects d'un Pointeur

Comme toute variable, un pointeur possède deux aspects :

Les deux aspects d'un pointeur

Adresse



- Valeur de p : 0x1000 (l'adresse de x)
- Adresse de p (&p) : 0x2000
- Valeur pointée (*p) : 42

```
int x = 42;           // Variable entière
int *p = &x;          // p pointe vers x
printf("%d", *p);     // Affiche 42
```

4.3 Opérateurs sur les Pointeurs

Opérateurs fondamentaux

&x	Opérateur "adresse de"	Retourne l'adresse de la variable x
*p	Opérateur de déréférencement	Accède à la valeur à l'adresse p
p + n	Addition d'un entier	Avance de $n \times \text{sizeof}(*p)$ octets
p - n	Soustraction d'un entier	Reculé de $n \times \text{sizeof}(*p)$ octets
p - q	Différence de pointeurs	Nombre d'éléments entre p et q

4.4 Exemple Pas à Pas : Évolution de la Mémoire

Suivons l'exécution du code suivant sur une machine 32 bits little-endian :

```
1 int x = 0x3CD02700; /* &x = 0x04 */
2 int y = x + 3;      /* &y = 0x18, y = 0x3CD02703 */
3 int *z;             /* &z = 0x20 */
4 z = &y + 3;         /* z = 0x18 + 3*4 = 0x24 */
5 *z = y;             /* Écrit y à l'adresse 0x24 */
```

Étape 0: État initial

```
int x = 0x3CD02700; // &x = 0x04
int y = x + 3;      // &y = 0x18, y = 0x3CD02703
int *z;             // &z = 0x20
z = &y + 3;         // z = 0x18 + 3*4 = 0x24
*z = y;             // Écrit y à l'adresse 0x24
```

Addr	+0	+1	+2	+3	Variable
0x00					
0x04					
0x08					
0x0C					
0x10					
0x14					
0x18					
0x1C					
0x20					
0x24					

(Mémoire non initialisée)

Étape 1: int x = 0x3CD02700;

```
int x = 0x3CD02700; // &x = 0x04
int y = x + 3;      // &y = 0x18, y = 0x3CD02703
int *z;             // &z = 0x20
z = &y + 3;         // z = 0x18 + 3*4 = 0x24
*z = y;             // Écrit y à l'adresse 0x24
```

Addr	+0	+1	+2	+3	Variable
0x00					
0x04	00	27	D0	3C	x = 0x3CD02700
0x08					
0x0C					
0x10					
0x14					
0x18					
0x1C					
0x20					
0x24					

x est initialisé à 0x3CD02700
(stocké en little-endian : 00 27 D0 3C)

Étape 2: int y = x + 3;

```
int x = 0x3CD02700; // &x = 0x04
int y = x + 3;      // &y = 0x18, y = 0x3CD02703
int *z;             // &z = 0x20
z = &y + 3;         // z = 0x18 + 3*4 = 0x24
*z = y;             // Écrit y à l'adresse 0x24
```

Addr	+0	+1	+2	+3	Variable
0x00					
0x04	00	27	D0	3C	x = 0x3CD02700
0x08					
0x0C					
0x10					
0x14					
0x18	03	27	D0	3C	y = 0x3CD02703
0x1C					
0x20					
0x24					

$y = x + 3 = 0x3CD02700 + 3 = 0x3CD02703$

Étape 3: int *z;

```
int x = 0x3CD02700; // &x = 0x04
int y = x + 3;      // &y = 0x18, y = 0x3CD02703
int *z;             // &z = 0x20
z = &y + 3;         // z = 0x18 + 3*4 = 0x24
*z = y;             // Écrit y à l'adresse 0x24
```

Addr	+0	+1	+2	+3	Variable
0x00					
0x04	00	27	D0	3C	x = 0x3CD02700
0x08					
0x0C					
0x10					
0x14					
0x18	03	27	D0	3C	y = 0x3CD02703
0x1C					
0x20	??	??	??	??	z = ??? (non init.)
0x24					

z est déclaré mais NON initialisé
(contient une valeur indéterminée)

Étape 4: z = &y + 3;

```
int x = 0x3CD02700; // &x = 0x04
int y = x + 3;      // &y = 0x18, y = 0x3CD02703
int *z;             // &z = 0x20
z = &y + 3;         // z = 0x18 + 3*4 = 0x24
*z = y;             // Écrit y à l'adresse 0x24
```

Addr	+0	+1	+2	+3	Variable
0x00					
0x04	00	27	D0	3C	x = 0x3CD02700
0x08					
0x0C					
0x10					
0x14					
0x18	03	27	D0	3C	y = 0x3CD02703
0x1C					
0x20	24	00	00	00	z = 0x24
0x24					

$z = \&y + 3 = 0x18 + 3 \times 4 = 0x18 + 0x0C = 0x24$
(arithmétique des pointeurs : +3 éléments int = +12 octets)

Étape 5: *z = y;

```
int x = 0x3CD02700; // &x = 0x04
int y = x + 3;      // &y = 0x18, y = 0x3CD02703
int *z;             // &z = 0x20
z = &y + 3;         // z = 0x18 + 3*4 = 0x24
*z = y;             // Écrit y à l'adresse 0x24
```

Addr	+0	+1	+2	+3	Variable
0x00					
0x04	00	27	D0	3C	x = 0x3CD02700
0x08					
0x0C					
0x10					
0x14					
0x18	03	27	D0	3C	y = 0x3CD02703
0x1C					
0x20	24	00	00	00	z = 0x24
0x24	03	27	D0	3C	*z = 0x3CD02703

*z = y écrit la valeur de y (0x3CD02703)
à l'adresse pointée par z (0x24)

L'instruction $z = \&y + 3$ illustre l'arithmétique des pointeurs : on ajoute 3 à un pointeur vers `int`, ce qui ajoute $3 \times 4 = 12$ octets à l'adresse. Ainsi, $0x18 + 0x0C = 0x24$.

5 Les Tableaux : Zones Mémoire Contiguës

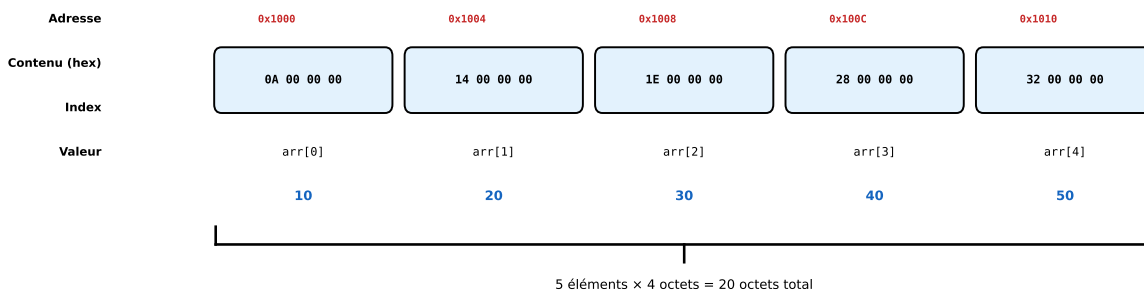
5.1 Définition et Représentation en Mémoire

Tableau

Un **tableau** est une séquence d'emplacements mémoire **consécutifs** stockant des éléments de même type. Les éléments sont indexés de 0 à $n - 1$ où n est la taille du tableau. Un tableau ne connaît pas sa propre taille.

```
1 int arr[5] = {10, 20, 30, 40, 50};
2 /* Si &arr[0] = 0x1000, alors :
3    arr[0] est à 0x1000, arr[1] à 0x1004, arr[2] à 0x1008, etc. */
```

Tableau int arr[5] en mémoire (32 bits, little-endian)



Formule : $\&arr[i] = \&arr[0] + i \times \text{sizeof}(\text{int})$
Exemple : $\&arr[3] = 0x1000 + 3 \times 4 = 0x100C$

5.2 Absence de Contrôle de Débordement

Danger : Pas de vérification des bornes

Le C ne vérifie **jamais** si un accès tableau est dans les limites. Un accès hors bornes compile sans erreur et s'exécute, mais corrompt la mémoire ou provoque un crash.

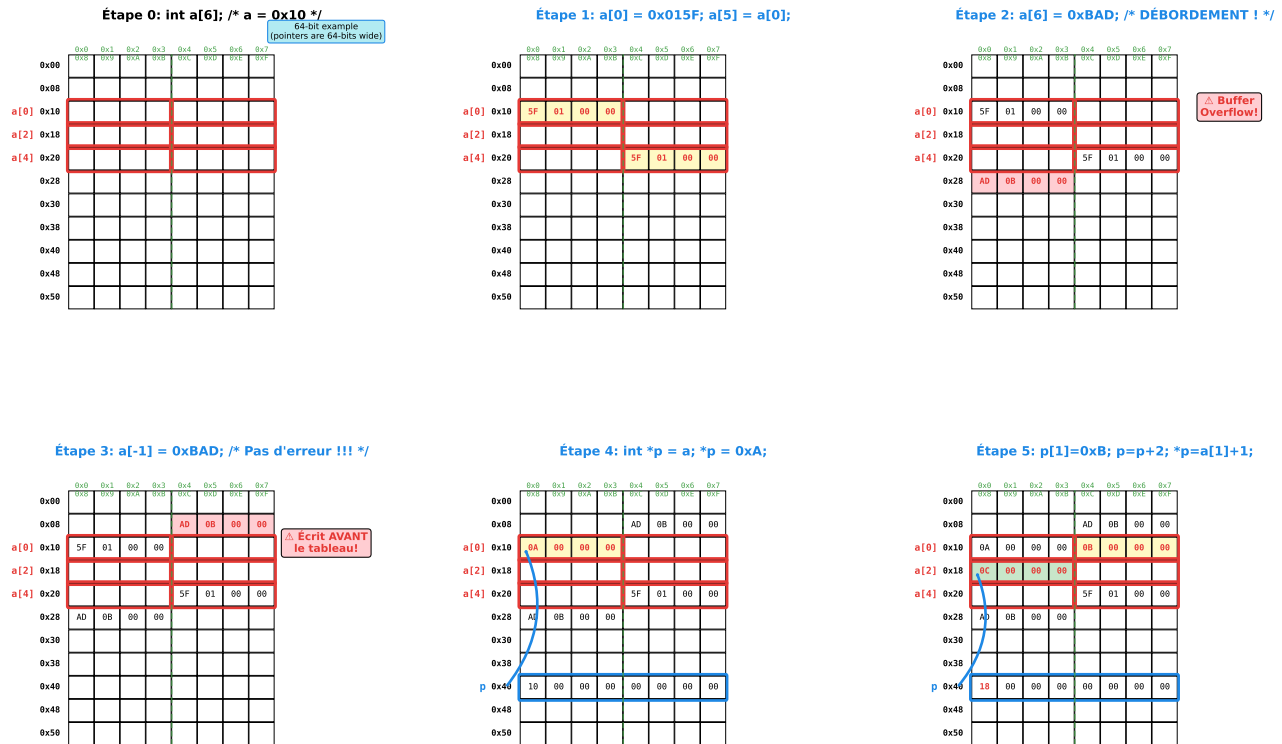
```
1 int notes[20];
2 for (int i = 0; i < 40; i++) /* Boucle jusqu'à 40 au lieu de 20 ! */
3     notes[i] = 0;           /* Écrase 80 octets au-delà du tableau */
4 /* Ce code compile sans warning et peut corrompre la pile */
```

Les conséquences d'un débordement dépendent de ce qui se trouve après le tableau en mémoire : autres variables, adresse de retour, etc. C'est une source majeure de vulnérabilités de sécurité (buffer overflow).

Exemple pas à pas : Tableaux

La figure suivante illustre l'évolution de la mémoire lors de manipulations de tableaux et de pointeurs, incluant des accès hors bornes dangereux.

Évolution mémoire : Tableaux et débordements (64 bits)



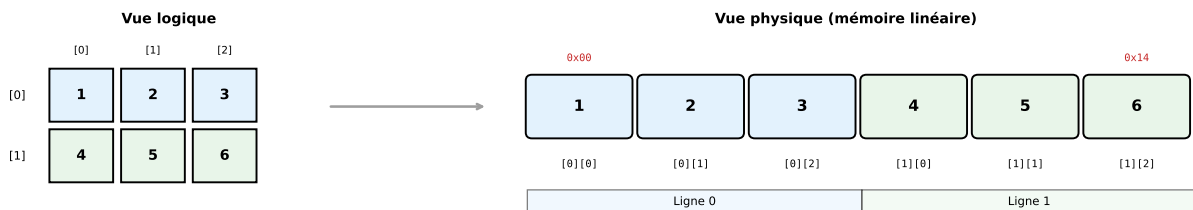
- Étapes 0-1 : Déclaration de `int a[6]` et affectations légales
- Étape 2 : `a[6] = 0xBAD` écrit **après** le tableau (buffer overflow)
- Étape 3 : `a[-1] = 0xBAD` écrit **avant** le tableau (aucune erreur!)
- Étapes 4-5 : Manipulation via pointeur `p` et arithmétique des pointeurs

5.3 Tableaux à Plusieurs Dimensions

Un tableau 2D est stocké en mémoire de manière linéaire, ligne par ligne (row-major order).

```
1 int mat[2][3] = {{1, 2, 3}, {4, 5, 6}};
2 /* En mémoire : 1, 2, 3, 4, 5, 6 (consécutifs) */
```

Tableau `mat[2][3]` en mémoire (row-major order)



Formule : `&mat[i][j] = base + (i × nb_colonnes + j) × sizeof(int)`

6 Relation Fondamentale Pointeurs-Tableaux

6.1 Conversion Tableau vers Pointeur

En C, le nom d'un tableau est automatiquement converti en pointeur vers son premier élément dans la plupart des contextes. Cette conversion est appelée *array decay*.

Équivalences fondamentales

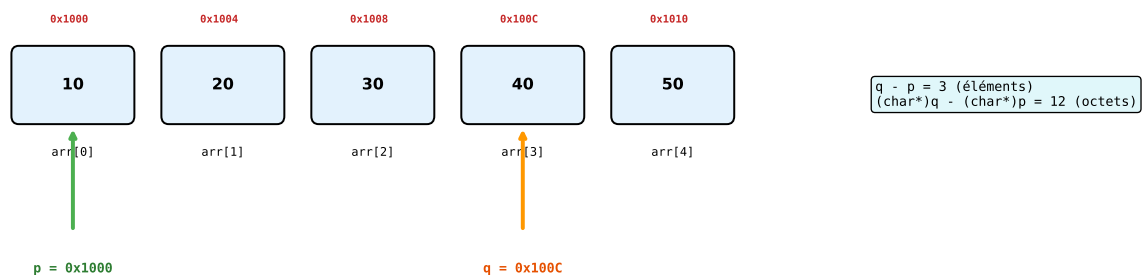
```
arr    ≡ &arr[0]
arr + i ≡ &arr[i]
*(arr + i) ≡ arr[i]
p[i]   ≡ *(p + i)
```

Ces équivalences expliquent pourquoi la syntaxe `i[arr]` est valide en C : elle est interprétée comme `*(i + arr)`, ce qui est identique à `*(arr + i) = arr[i]`.

```
1 int arr[5] = {10, 20, 30, 40, 50};
2 int *p = arr; /* Équivalent à : int *p = &arr[0]; */
3
4 /* Toutes ces expressions sont équivalentes et valent 30 : */
5 printf("%d\n", arr[2]); /* Notation tableau */
6 printf("%d\n", *(arr + 2)); /* Arithmétique pointeur */
7 printf("%d\n", p[2]); /* Pointeur avec notation tableau */
8 printf("%d\n", *(p + 2)); /* Pointeur avec déréférencement */
9 printf("%d\n", 2[arr]); /* Syntaxe exotique mais valide ! */
```

6.2 Visualisation de Pointeurs sur un Tableau

Pointeurs p et q sur le tableau arr



6.3 Différences entre Tableau et Pointeur

Bien que très similaires dans leur utilisation, tableau et pointeur ne sont pas identiques :

Différences entre tableau et pointeur

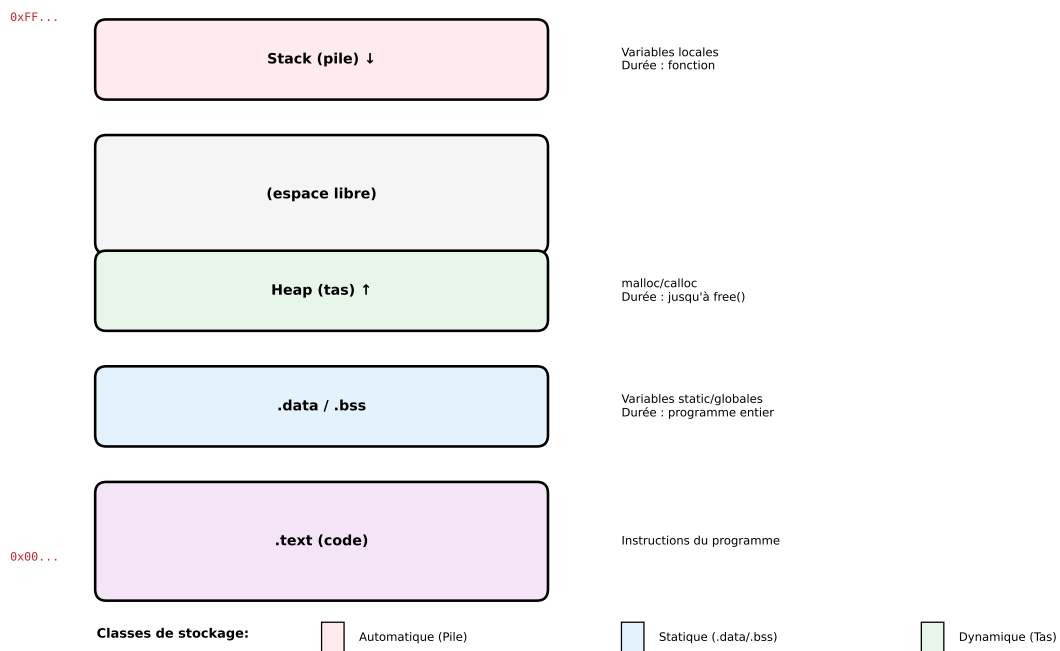
	<code>int arr[5]</code>	<code>int *p</code>
sizeof	20 (taille totale)	4 ou 8 (taille pointeur)
&	Adresse du tableau	Adresse de la variable p
Modifiable	Non (arr = ... interdit)	Oui (p = ... autorisé)
Allocation	Automatique/statique	Variable séparée

7 Classes de Stockage et Durée de Vie des Tableaux

7.1 Les Trois Classes de Stockage

En C, un tableau peut être stocké dans trois zones mémoire différentes, chacune avec ses caractéristiques propres.

Organisation mémoire d'un processus



Comparaison des classes de stockage

Classe	Zone	Durée de vie	Taille
Automatique	Pile (stack)	Durée du bloc {}	Fixe à la compilation
Statique	Segment data	Toute l'exécution	Fixe à la compilation
Dynamique	Tas (heap)	Jusqu'à <code>free()</code>	Variable à l'exécution

```

1 int global[100];           /* Statique : segment data, existe toujours */
2
3 void exemple(int n) {
4     int local[10];         /* Automatique : sur la pile, détruit à la fin */
5     static int persistant[5]; /* Statique : segment data, conservé entre appels */
6     int *dynamique = malloc(n * sizeof(int)); /* Dynamique : sur le tas */
7
8     /* ... utilisation ... */
9 }
```

```

10     free(dynamique);           /* Libération obligatoire */
11 } /* local est détruit ici, persistant survit */

```

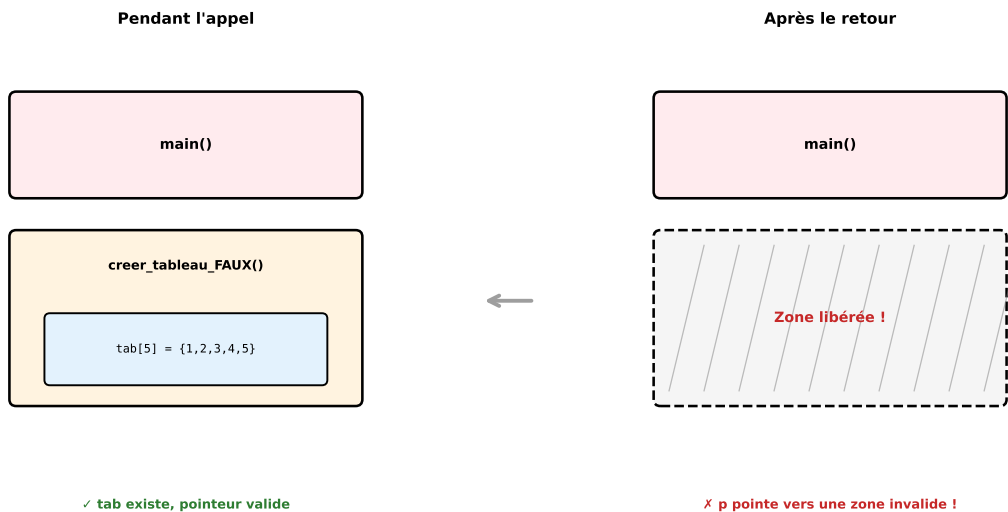
7.2 Le Problème du Retour de Tableau

Une erreur classique consiste à retourner un pointeur vers un tableau local (automatique).

```

1 /* ERREUR : retourne un pointeur vers une zone mémoire invalide */
2 int *creer_tableau_FAUX(void) {
3     int tab[5] = {1, 2, 3, 4, 5}; /* Alloué sur la pile */
4     return tab; /* Retourne l'adresse de tab */
5 } /* tab est détruit ici ! Le pointeur retourné est invalide */
6
7 int main(void) {
8     int *p = creer_tableau_FAUX();
9     printf("%d\n", p[0]); /* Comportement indéfini ! */
10    return 0;
11 }

```



7.3 Solutions pour Retourner un Tableau

Solution 1 : Tableau statique – La fonction retourne un pointeur vers un tableau qui persiste.

```

1 /* OK mais attention : un seul tableau partagé par tous les appels */
2 int *creer_tableau_statique(void) {
3     static int tab[5] = {1, 2, 3, 4, 5}; /* Segment data */
4     return tab; /* Valide : tab existe toujours */
5 }
6
7 int main(void) {
8     int *p = creer_tableau_statique();
9     printf("%d\n", p[0]); /* OK : affiche 1 */
10
11     /* ATTENTION : tous les appels partagent le même tableau */
12     int *q = creer_tableau_statique();
13     q[0] = 99;
14     printf("%d\n", p[0]); /* Affiche 99 ! p et q pointent au même endroit */
15     return 0;
16 }

```

Solution 2 : Allocation dynamique – Chaque appel crée un nouveau tableau.

```

1 /* OK : chaque appel alloue un nouveau tableau */
2 int *creer_tableau_dynamique(int n) {
3     int *tab = malloc(n * sizeof(int)); /* Sur le tas */
4     if (tab == NULL) return NULL; /* Vérifier l'allocation */
5     for (int i = 0; i < n; i++)
6         tab[i] = i + 1;
7     return tab; /* Valide : le tas n'est pas libéré au retour */
8 }
9
10 int main(void) {
11     int *p = creer_tableau_dynamique(5);
12     int *q = creer_tableau_dynamique(5); /* Deux tableaux distincts */
13
14     q[0] = 99;

```

```

15     printf("%d\n", p[0]); /* Affiche 1 : p et q sont indépendants */
16
17     free(p); /* OBLIGATOIRE : libérer la mémoire */
18     free(q);
19     return 0;
20 }

```

Solution 3 : Passage par paramètre – L'appelant fournit le tableau.

```

1  /* L'appelant gère l'allocation */
2  void remplir_tableau(int *tab, int n) {
3      for (int i = 0; i < n; i++)
4          tab[i] = i + 1;
5  }
6
7  int main(void) {
8      int local[5]; /* Automatique */
9      remplir_tableau(local, 5); /* La fonction remplit le tableau fourni */
10     printf("%d\n", local[0]); /* OK : affiche 1 */
11     return 0;
12 }

```

8 Allocation Dynamique : malloc, calloc, realloc, free

8.1 Principe de l'Allocation Dynamique

L'allocation dynamique permet de créer des zones mémoire dont la taille est déterminée à l'exécution. Ces zones sont allouées sur le **tas** (heap) et persistent jusqu'à leur libération explicite.

Fonctions d'allocation (<stdlib.h>)

malloc(size)	Alloue size octets (non initialisés)
calloc(n, size)	Alloue n × size octets (initialisés à 0)
realloc(ptr, size)	Redimensionne la zone pointée par ptr
free(ptr)	Libère la zone pointée par ptr

```

1  #include <stdlib.h>
2
3  /* Allocation d'un tableau de n entiers */
4  int *tab = malloc(n * sizeof(int)); /* n * 4 octets (si int = 4) */
5
6  /* TOUJOURS vérifier le retour de malloc */
7  if (tab == NULL) {
8      fprintf(stderr, "Erreur allocation\n");
9      return 1;
10 }
11
12 /* Utilisation */
13 for (int i = 0; i < n; i++)
14     tab[i] = i * 10;
15
16 /* Libération OBLIGATOIRE */
17 free(tab);
18 tab = NULL; /* Bonne pratique : évite les accès après free */

```

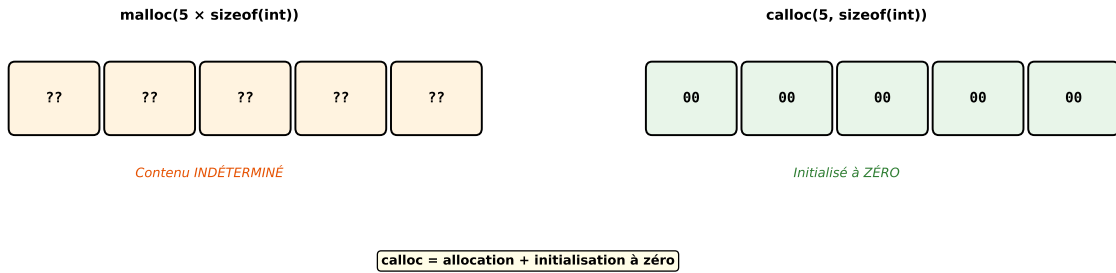
8.2 malloc vs calloc

```

1  /* malloc : mémoire NON initialisée (contient des valeurs aléatoires) */
2  int *p = malloc(5 * sizeof(int));
3  /* p[0], p[1], ... contiennent des valeurs indéterminées */
4
5  /* calloc : mémoire initialisée à ZÉRO */
6  int *q = calloc(5, sizeof(int));
7  /* q[0] = q[1] = ... = q[4] = 0 */

```

malloc vs calloc



8.3 Redimensionnement avec realloc

```
1 int *tab = malloc(5 * sizeof(int));
2 /* ... remplissage de tab[0] à tab[4] ... */
3
4 /* Agrandir à 10 éléments */
5 int *nouveau = realloc(tab, 10 * sizeof(int));
6 if (nouveau == NULL) {
7     /* Échec : tab est toujours valide, mais pas agrandi */
8     free(tab);
9     return 1;
10 }
11 tab = nouveau; /* Succès : utiliser le nouveau pointeur */
12 /* tab[0]..tab[4] conservés, tab[5]..tab[9] non initialisés */
13
14 free(tab);
```

8.4 Erreurs Classiques

```
1 /* 1. Oublier de vérifier malloc */
2 int *p = malloc(1000000000000); /* Peut échouer ! */
3 p[0] = 42; /* Crash si p == NULL */
4
5 /* 2. Oublier free (fuite mémoire) */
6 void fuite(void) {
7     int *p = malloc(100);
8     /* ... pas de free(p) ... */
9 } /* Mémoire perdue à chaque appel */
10
11 /* 3. Double free */
12 int *p = malloc(10);
13 free(p);
14 free(p); /* Comportement indéfini ! */
15
16 /* 4. Accès après free (use-after-free) */
17 int *p = malloc(10);
18 free(p);
19 p[0] = 42; /* Comportement indéfini ! */
```

9 Tableaux à Longueur Variable (VLA)

9.1 Syntaxe et Utilisation

Depuis C99, il est possible de déclarer des tableaux dont la taille est une variable (et non une constante).

```
1 void traiter(int n) {
2     int tab[n]; /* VLA : taille déterminée à l'exécution */
3
4     for (int i = 0; i < n; i++)
5         tab[i] = i * i;
6
7     /* tab est automatique : détruit à la fin de la fonction */
8 }
9
10 int main(void) {
11     int taille;
12     printf("Taille : ");
13     scanf("%d", &taille);
14
15     int donnees[taille]; /* VLA : alloué sur la pile */
16     /* ... */
17     return 0;
18 }
```

9.2 VLA vs Allocation Dynamique

VLA vs Allocation Dynamique

	int tab[n] (VLA)	malloc(n × sizeof(int))
Zone mémoire	Pile (stack)	Tas (heap)
Libération	Automatique	Manuelle (free)
Taille max	Limitée (~8 Mo)	Grande (~Go)
Standard C	C99+ (optionnel C11+)	C89+
Redimensionnable	Non	Oui (realloc)

Limitations des VLA

- Les VLA sont alloués sur la **pile**, dont la taille est limitée (typiquement 8 Mo). Un VLA trop grand provoque un *stack overflow*.
- Les VLA ne peuvent pas être **static** ni globaux.
- Les VLA sont optionnels depuis C11 (macro `__STDC_NO_VLA__`).
- Pour de grands tableaux ou une durée de vie contrôlée, préférer `malloc`.

```
1  /* Danger : VLA trop grand */
2  void danger(int n) {
3      int enorme[n]; /* Si n = 10000000, stack overflow probable */
4  }
5
6  /* Solution : utiliser malloc pour les grands tableaux */
7  void safe(int n) {
8      int *tab = malloc(n * sizeof(int));
9      if (tab == NULL) { /* gérer l'erreur */ }
10     /* ... */
11     free(tab);
12 }
```

10 Arithmétique des Pointeurs en Détail

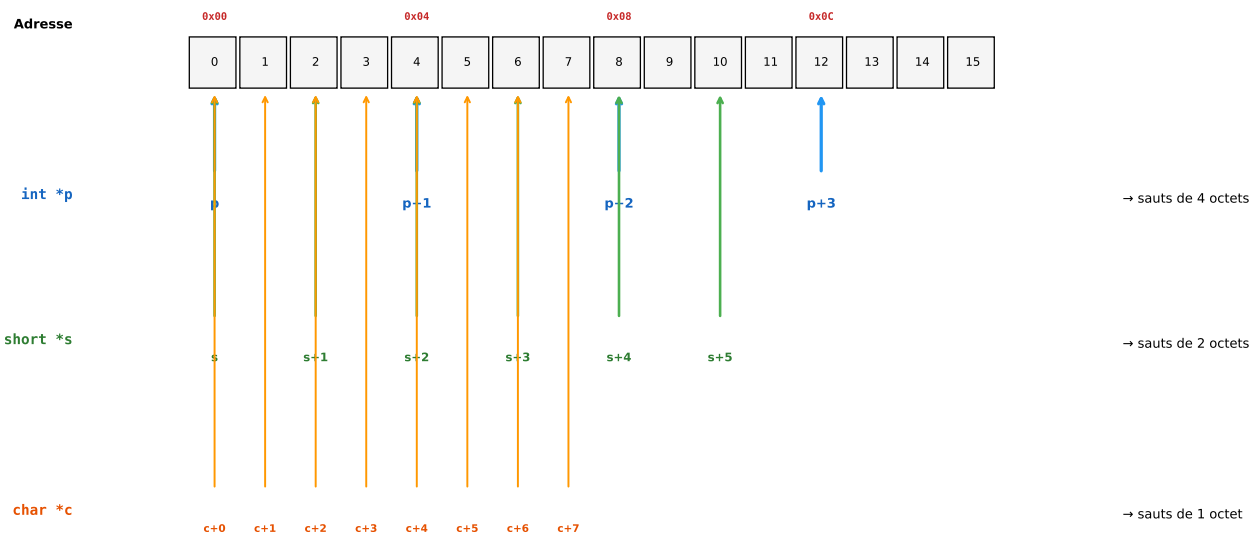
10.1 Règle Fondamentale

Arithmétique dimensionnée

Toute opération arithmétique sur un pointeur est **dimensionnée** par la taille du type pointé. Ajouter n à un pointeur de type T^* ajoute $n \times \text{sizeof}(T)$ octets à l'adresse.

Cette règle permet de parcourir un tableau naturellement : $p + 1$ pointe toujours vers l'élément suivant, quelle que soit la taille des éléments.

Arithmétique selon le type du pointeur



$p + n$ ajoute $n \times \text{sizeof}(*p)$ octets à l'adresse

10.2 Opérations Valides et Exemples

```

1 int arr[5] = {10, 20, 30, 40, 50}; /* &arr[0] = 0x1000 */
2 int *p = arr; /* p = 0x1000 */
3 int *q = &arr[3]; /* q = 0x100C */
4
5 /* Addition et soustraction d'entiers */
6 printf("p + 2 = %p\n", (void*)(p + 2)); /* 0x1008 (+8 octets) */
7 printf("q - 1 = %p\n", (void*)(q - 1)); /* 0x1008 (-4 octets) */
8
9 /* Différence de pointeurs : résultat en ÉLÉMENTS, pas en octets */
10 printf("q - p = %ld\n", q - p); /* 3 (éléments) */
11
12 /* Pour obtenir la différence en octets, cast vers char* */
13 printf("(char*)q - (char*)p = %ld\n", (char*)q - (char*)p); /* 12 octets */
14
15 /* Comparaisons */
16 if (p < q) printf("p est avant q en mémoire\n");
17
18 /* Incrémentation */
19 p++; /* p passe de 0x1000 à 0x1004 */

```

10.3 Réinterprétation Mémoire par Cast

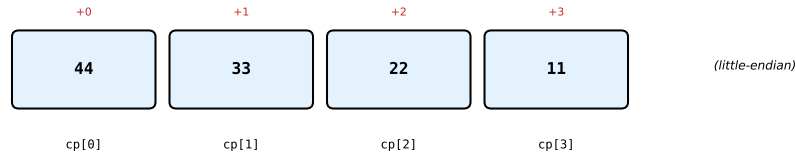
Un cast de pointeur permet de réinterpréter une zone mémoire avec un type différent. C'est utile pour examiner la représentation interne des données.

```

1 int x = 0x11223344;
2 char *cp = (char *)&x; /* Vue de x comme séquence d'octets */
3
4 printf("Octets de x en mémoire:\n");
5 for (int i = 0; i < 4; i++) {
6     printf("    octet %d: 0x%02X\n", i, (unsigned char)cp[i]);
7 }
8 /* Sur little-endian : 44 33 22 11 */
9 /* Sur big-endian   : 11 22 33 44 */

```

Réinterprétation de 0x11223344 via char*



Le cast (char*) permet de lire octet par octet sans changer le contenu de la mémoire

11 Les Chaînes de Caractères

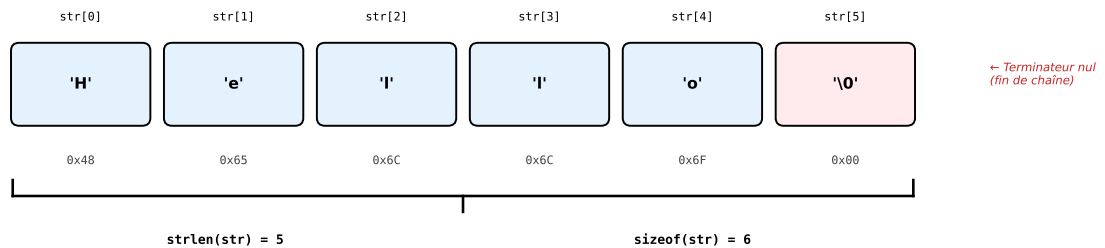
11.1 Définition et Représentation

Chaîne de caractères en C

Une **chaîne de caractères** est un tableau de **char** terminé par le caractère nul `'\0'` (code ASCII 0x00). Ce caractère de terminaison permet aux fonctions de déterminer où finit la chaîne.

```
1 char str1[] = "Hello"; /* Tableau de 6 chars : H e l l o \0 */
2 char str2[10] = "Hi"; /* Tableau de 10 chars : H i \0 ? ? ? ? ? ? */
3 char str3[] = {'A', 'B', 0}; /* Équivalent à "AB" */
```

Représentation de "Hello" en mémoire



11.2 Table ASCII Partielle

Les caractères sont stockés selon leur code ASCII (American Standard Code for Information Interchange).

Table ASCII partielle

Hex	Char	Hex	Char	Hex	Char	Hex	Char
00	NUL	30	0	41	A	61	a
09	TAB	31	1	42	B	62	b
0A	LF	32	2	43	C	63	c
0D	CR	39	9	5A	Z	7A	z
20	SPC	3F	?				

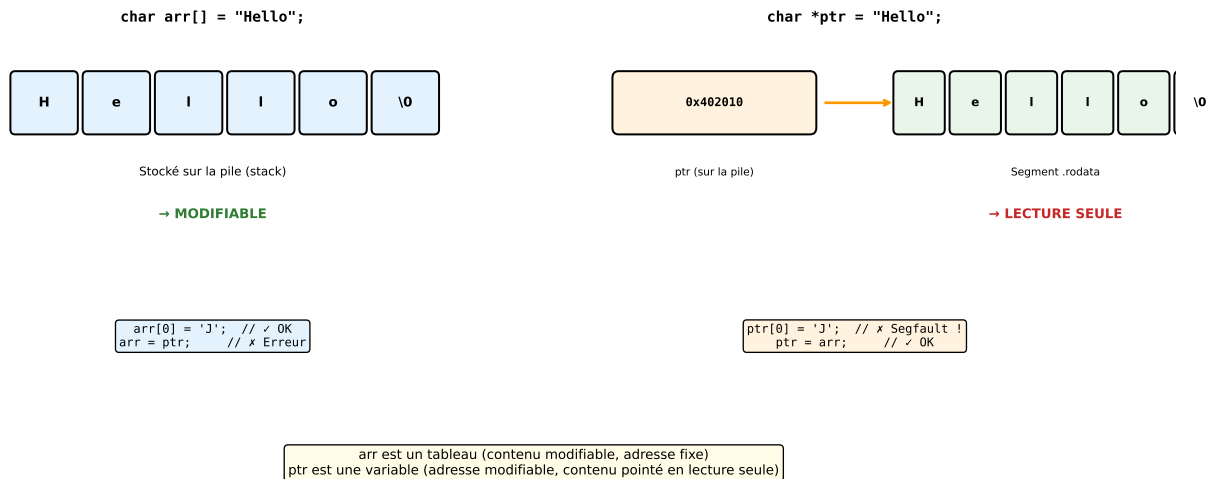
'A'=0x41, 'a'=0x61, différence=0x20 | Chiffres '0'-'9' : 0x30-0x39

Relations utiles : 'A' = 0x41, 'a' = 0x61, différence = 0x20. Les chiffres '0' à '9' sont aux codes 0x30 à 0x39.

11.3 Différence Capitale : Tableau vs Pointeur sur Chaîne

```
1 char arr[] = "Hello"; /* Tableau alloué sur la pile, MODIFIABLE */
2 char *ptr = "Hello"; /* Pointeur vers le segment de données, LECTURE SEULE */
3
4 arr[0] = 'J'; /* OK : modifie la copie locale -> "Jello" */
5 ptr[0] = 'J'; /* ERREUR : segfault ! Le segment données est protégé */
6
7 arr = ptr; /* ERREUR de compilation : arr n'est pas modifiable */
8 ptr = arr; /* OK : ptr peut pointer vers une autre adresse */
```

Tableau vs Pointeur sur chaîne



11.4 Fonctions Standard de Manipulation

Les fonctions de <string.h> opèrent sur des chaînes terminées par '\0'.

Fonctions de <string.h>

strlen(s)	Longueur de s	Ne compte pas le '\0'
strcpy(dst, src)	Copie src dans dst	Danger si dst trop petit
strncpy(dst, src, n)	Copie au plus n chars	Plus sûr
strcat(dst, src)	Concatène src à dst	Danger si dst trop petit
strcmp(s1, s2)	Compare s1 et s2	0 si égales, <0 si s1<s2, >0 si s1>s2
strchr(s, c)	Cherche c dans s	Retourne pointeur ou NULL

```
1 #include <string.h>
2
3 char dest[20];
4 strcpy(dest, "Hello"); /* dest = "Hello" */
5 strcat(dest, " World"); /* dest = "Hello World" */
6 printf("Longueur: %zu\n", strlen(dest)); /* 11 */
7 printf("Taille: %zu\n", sizeof(dest)); /* 20 */
8
9 if (strcmp(dest, "Hello World") == 0)
10 printf("Chaînes identiques\n");
```

12 Visualisation de la Mémoire en C

Le programme suivant permet d'examiner la représentation mémoire des données.

Listing 2 – Programme complet de visualisation mémoire

```
1 #include <stdio.h>
2 #include <stdint.h>
3
```

```

4 typedef unsigned char *byte_pointer;
5
6 /* Affiche len octets à partir de start, 2 caractères par octet */
7 void show_bytes(byte_pointer start, size_t len) {
8     printf(" ");
9     for (size_t i = 0; i < len; i++)
10         printf("%.2x ", start[i]);
11     printf("\n");
12 }
13
14 void show_int(int x) {
15     printf("int %d (0x%X):\n", x, x);
16     show_bytes((byte_pointer)&x, sizeof(int));
17 }
18
19 void show_float(float x) {
20     printf("float %f:\n", x);
21     show_bytes((byte_pointer)&x, sizeof(float));
22 }
23
24 void show_pointer(void *x) {
25     printf("pointer %p:\n", x);
26     show_bytes((byte_pointer)&x, sizeof(void *));
27 }
28
29 int main(void) {
30     int val = 0x12345678;
31     float f = 1.0f;
32
33     show_int(val);
34     show_float(f);
35     show_pointer(&val);
36
37     /* Vérification endianness */
38     printf("\nCette machine est ");
39     uint32_t test = 1;
40     if (*(uint8_t*)&test == 1)
41         printf("Little-Endian\n");
42     else
43         printf("Big-Endian\n");
44
45     return 0;
46 }

```

Sortie typique (64 bits)

```

int 305419896 (0x12345678):
  78 56 34 12
float 1.000000:
  00 00 80 3f
pointer 0x7ffd5c9fa9ac:
  ac a9 9f 5c fd 7f 00 00

Cette machine est Little-Endian

```

13 Récapitulatif du Chapitre

Points clés à retenir

1. Le **mot machine** (32 ou 64 bits) détermine la taille des adresses et des pointeurs
2. La mémoire est un tableau linéaire d'octets; chaque octet a une adresse unique
3. L'**adresse d'un objet** est l'adresse de son premier octet
4. **Endianness** : Little-Endian (Intel) stocke le LSB à l'adresse basse; Big-Endian fait l'inverse
5. Un **tableau** occupe des cases mémoire consécutives; le C ne vérifie pas les débordements
6. Le **nom d'un tableau** est converti en pointeur vers son premier élément
7. **Équivalences** : $\text{arr}[i] \equiv *(\text{arr}+i)$ et $\&\text{arr}[i] \equiv \text{arr}+i$
8. L'**arithmétique des pointeurs** est dimensionnée par `sizeof(type pointé)`
9. Une **chaîne** est un tableau de `char` terminé par `'\0'`
10. **Tableau vs pointeur sur chaîne** : le tableau est modifiable, le littéral ne l'est pas

Formules et syntaxes essentielles

Adresse de <code>arr[i]</code>	<code>&arr[0] + i * sizeof(arr[0])</code>
<code>ptr + n</code>	Ajoute <code>n * sizeof(*ptr)</code> octets
<code>ptr1 - ptr2</code>	Nombre d'éléments (pas d'octets)
<code>strlen(s)</code>	Nombre de caractères avant <code>'\0'</code>
<code>sizeof(arr)</code>	Taille totale du tableau en octets
<code>sizeof(ptr)</code>	Taille du pointeur (4 ou 8)